

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)A C
- B)A
- C)B
- D)B C

Q2. What will be printed?

```
class InvalidAgeError(Exception):
    pass

def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"
```

```
try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

A)InvalidAgeError
15
B)InvalidAgeError
Age is too low
C)Nothing, because custom exceptions
cannot be caught
D)Exception
Age is too low

Q3. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]

result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

- A)10
- B)24
- C)0
- D)9

Q4. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)5
- B)None
- C)6
- D)3

Q5. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

- A)10
- B)RuntimeError
- C)20
- D)None

Q6. What will be printed?

```
x = 10
```



```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)30

B)NameError

C)20

D)10

Q7. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A)Error

B)10

C)5

D)func(5)

Q8. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)A

B)B

C)A and B

D)Nothing

Q9. What is the output?

```
def fun(a=10, b=20):
```

```
    return a - b
```

```
print(fun(30))
```

A)10

B)20

C)30

D)-10

Q10. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

A)Both import only sqrt

B)First provides the module namespace; second imports sqrt directly

C)There is no difference

D)First imports only sqrt

Q11. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)A, and execution ends normally

B)A, followed by propagation of the same exception

C)No Output

D)B

Q12. What is the output?

```
def fun():
```

```
    try:
```

```
        print("A")
```

```
        raise ValueError
```

```
    except ValueError:
```

```
        print("B")
```

```
        return 10
```

```
    else:
```

```
        print("C")
```

```
        return 20
```

```
    finally:
```

```
        print("D")
```

```
print(fun())
```

A)A

B



10
D
B)A
B
D
10
C)A
B
C
D
10
D)A
C
D
20

Q13. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

A)Error
B)[10]
[10, 10]
C)[10, 10]
[10]
D)[10]
[10]

Q14. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()  
A)SyntaxError  
B)NameError  
C)10
```

D)20

Q15. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)TypeError

B)MyError

C)Something went wrong

D)Exception

Q16. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```

A)Finally only

B)10 only

C)Finally followed by 10

D)10 followed by Finally

Q17. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)5.0

B)5

C)sqrt(25)

D)25

Q18. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,
```



numbers)

)

)

print(result)

A)[2, 4]

B)[2, 4, 6, 8, 10]

C)[1, 3, 5]

D)[4, 8]

Q19. What will be printed?

def fun(a, b):

return a + b

print(fun(10, b=20))

A)20

B)30

C)SyntaxError

D)TypeError

Q20. What will happen?

def fun(a, b=10):

return a + b

print(fun(b=5, a=20))

A)TypeError

B)15

C)SyntaxError

D)25

Q21. What is the output?

def outer():

x = 10

def inner():

return x + 5

return inner

f = outer()

print(f())

A)None

B)NameError

C)10

D)15

Q22. Why is

try:

print(10 / 0)

except Exception:

print("A")

except ZeroDivisionError:

print("B")

codes's second handler never reached?

A)Exception is a superclass of ZeroDivisionError

B)except blocks execute randomly

C)ZeroDivisionError occurs before Exception

D)ZeroDivisionError is not an exception

Q23. What will be the output?

x = 10

def outer():

x = 20

def inner():

global x

x = 30

inner()

print(x)

outer()

print(x)

A)20

30

B)20

10

C)30

20

D)30

30

Q24. What will be the output?

x = 10

def fun():

print(x)

x = 20

fun()

A)UnboundLocalError

B)NameError

C)20



D)10

Q25. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

A)Wrong Value

B)e

C)ValueError

D)Nothing

Q26. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

x = 5

y = fun(x)

print(x, y)

A)5 15

B)15 15

C)15 5

D)5 5

Q27. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)10

B)NameError

C)50

D)UnboundLocalError

Q28. Suppose module_a.py contains:

```
print("Module loaded")
```

value = 10

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

10

B)10

10

C)Module loaded

Module loaded

10

D)ImportError

Q29. What happens?

```
def fun(a, b):  
    return a + b
```

print(fun(10, a=20))

A)20

B)SyntaxError

C)TypeError

D)30

Q30. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

A)TypeError

B)3

C)RecursionError

D)6

Q31. What does the following function return?

```
def fun():  
    print("Hello")
```

x = fun()

print(x)

A)None

B>Hello None

C>Hello



D)Error

Q32. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

A)UnboundLocalError

B)15

C)10

D)5

Q33. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)None

B)NameError

C)0

D)TypeError

Q34. What happens when this code runs?

```
def first():
```

```
    second()
```

```
def second():
```

```
    third()
```

```
def third():
```

```
    raise RuntimeError("Failure")
```

```
try:
```

```
    first()
```

```
except RuntimeError:
```

```
    print("Handled")
```

A)TypeError

B)Handled

C)NameError

D)RuntimeError cannot propagate through functions

Q35. What will be printed?

```
def make_counter():
```

```
    count = 0
```

```
    def counter():
```

```
        nonlocal count
```

```
        count += 1
```

```
        return count
```

```
    return counter
```

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

2

3

4

B)1

1

1

1

C)1

2

1

3

D)0

1

0

2

Q36. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

```
fun()
```

A)Nothing

B)A B

C)A



D)B

Q37. What is the output?

try:

print(10 / 0)

except Exception:

print("A")

except ZeroDivisionError:

print("B")

A)A

B)SyntaxError

C)A B

D)B

Q38. if test.py has

print("A")

if __name__ == "__main__":

print("B")

If test.py is executed directly, what will be printed?

A)Nothing

B)B

C)A

D)A followed by B

Q39. What will normally happen?

import math

import math

A)The module is normally loaded once and reused

B)ImportError

C)SyntaxError

D)The module is normally loaded and executed twice

Q40. What is the output?

f = lambda x: x + 5

print(f(10))

A>Error

B)5

C)15

D)10

Q41. Suppose sample.py contains:

x = 10

def display():

print("Hello")

What will this print?

import sample

print(sample.x)

sample.display()

A)10

Hello

B)Only Hello

C)x

display

D>Error

Q42. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

A)20

B)None

C)NameError

D)10

Q43. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.path

B)sys.modules

C)sys.imports

D).__module__

Q44. What will be printed?

try:

x = int("10")

except ValueError:

print("A")

else:

print("B")

finally:

print("C")

A)B C

B)A C

C)B



D)A

Q45. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

- A)Python will always reject it
- B)The local file can shadow the intended standard-library module
- C)Python automatically renames the local file
- D)random cannot be imported

Q46. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)negative
cleanup
- B)cleanup
100
- C)cleanup
-5
- D)100
cleanup

Q47. What is the purpose of this statement?

if __name__ == "__main__":

- A)It prevents all imports
- B)It prevents syntax errors
- C)It identifies code intended to run when the file is executed directly
- D)It creates the main function

Q48. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:
```

```
print("A")  
except Exception:  
    print("B")
```

- A)Nothing
- B)B
- C)A B
- D)A

Q49. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A)It executes correctly
- B)NameError
- C)SyntaxError
- D)TypeError

Q50. What happens here?

```
from math import sqrt
```

print(sqrt(36))

- A)6
- B)NameError
- C)36
- D)6.0

